Getting started with CloudFormation

You can begin using CloudFormation through the AWS Management Console by creating a stack from an example template, which will help you learn the basics of stack creation. A *template* is a text file that defines all the resources within a stack. A *stack* is the deployment of a CloudFormation template. From a single template, you can create multiple stacks. Each stack contains a collection of AWS resources that can be managed as a single unit.

CloudFormation is a free service; however, you are charged for the AWS resources you include in your stacks at the current rates for each. For more information about AWS pricing, go to the detail page for each product on http://aws.amazon.com.

Topics

- How CloudFormation works
- Signing up for an AWS account
- Creating your first stack

How CloudFormation works

This topic describes how CloudFormation works and introduces you to the key concepts you'll need to know about as you use it.

Topics

- Key concepts
- How CloudFormation works
- Ways to get started with CloudFormation

Key concepts

When you use CloudFormation, you work with *templates* and *stacks*. You create templates to describe your AWS resources and their properties. Whenever you create a stack, CloudFormation provisions the resources that are described in your template.

Topics

- Templates
- Stacks
- Change sets

Templates

A CloudFormation template is a YAML or JSON formatted text file. You can save these files with any extension, such as `.yaml`, `.json`, `.template`, or `.txt`. CloudFormation uses these templates as blueprints for building your AWS resources. For example, in a template, you can describe an Amazon EC2 instance, such as the instance type, the AMI ID, block device mappings, and its Amazon EC2 key pair name. Whenever you create a stack, you also specify a template that CloudFormation uses to create whatever you described in the template.

For example, if you created a stack with the following template, CloudFormation provisions an instance with an `ami-0ff8a91507f77f867` AMI ID, `t2.micro` instance type, `testkey` key pair name, and an Amazon EBS volume.

YAML

```
AWSTemplateFormatVersion: 2010-09-09
Description: A sample template
Resources:
  MyEC2Instance:
    Type: 'AWS::EC2::Instance'
    Properties:
      ImageId: ami-0ff8a91507f77f867
      InstanceType: t2.micro
      KeyName: testkey
      BlockDeviceMappings:
        - DeviceName: /dev/sdm
          Ebs:
            VolumeType: io1
            Iops: 200
            DeleteOnTermination: false
            VolumeSize: 20
```

JSON

```
{
    "AWSTemplateFormatVersion": "2010-09-09",
```

```
    "Description": "A sample template",
    "Resources": {
        "MyEC2Instance": {
            "Type": "AWS::EC2::Instance",
            "Properties": {
                "ImageId": "ami-0ff8a91507f77f867",
                "InstanceType": "t2.micro",
                "KeyName": "testkey",
                "BlockDeviceMappings": [
                    {
                        "DeviceName": "/dev/sdm",
                        "Ebs": {
                            "VolumeType": "io1",
                            "Iops": 200,
                            "DeleteOnTermination": false,
                            "VolumeSize": 20
                        }
                    }
                ]
            }
        }
    }
}
```

You can also specify multiple resources in a single template and configure these resources to work together. For example, you can modify the previous template to include an Elastic IP address (EIP) and associate it with the Amazon EC2 instance, as shown in the following example:

YAML

```
AWSTemplateFormatVersion: 2010-09-09
Description: A sample template
Resources:
  MyEC2Instance:
    Type: 'AWS::EC2::Instance'
    Properties:
      ImageId: ami-0ff8a91507f77f867
      InstanceType: t2.micro
      KeyName: testkey
      BlockDeviceMappings:
        - DeviceName: /dev/sdm
          Ebs:
            VolumeType: io1
```

```
            Iops: 200
            DeleteOnTermination: false
            VolumeSize: 20
  MyEIP:
    Type: 'AWS::EC2::EIP'
    Properties:
      InstanceId: !Ref MyEC2Instance
```

JSON

```
{
    "AWSTemplateFormatVersion": "2010-09-09",
    "Description": "A sample template",
    "Resources": {
        "MyEC2Instance": {
            "Type": "AWS::EC2::Instance",
            "Properties": {
                "ImageId": "ami-0ff8a91507f77f867",
                "InstanceType": "t2.micro",
                "KeyName": "testkey",
                "BlockDeviceMappings": [
                    {
                        "DeviceName": "/dev/sdm",
                        "Ebs": {
                            "VolumeType": "io1",
                            "Iops": 200,
                            "DeleteOnTermination": false,
                            "VolumeSize": 20
                        }
                    }
                ]
            }
        },
        "MyEIP": {
            "Type": "AWS::EC2::EIP",
            "Properties": {
                "InstanceId": {
                    "Ref": "MyEC2Instance"
                }
            }
        }
    }
}
```

The previous templates are centered around a single Amazon EC2 instance; however, CloudFormation templates have additional capabilities that you can use to build complex sets of resources and reuse those templates in multiple contexts. For example, you can add input parameters whose values are specified when you create a CloudFormation stack. In other words, you can specify a value like the instance type when you create a stack instead of when you create the template, making the template easier to reuse in different situations.

Stacks

When you use CloudFormation, you manage related resources as a single unit called a stack. You create, update, and delete a collection of resources by creating, updating, and deleting stacks. All the resources in a stack are defined by the stack's CloudFormation template. Suppose you created a template that includes an Auto Scaling group, Elastic Load Balancing load balancer, and an Amazon Relational Database Service (Amazon RDS) database instance. To create those resources, you create a stack by submitting the template that you created, and CloudFormation provisions all those resources for you.

Change sets

If you need to make changes to the running resources in a stack, you update the stack. Before making changes to your resources, you can generate a change set, which is a summary of your proposed changes. Change sets allow you to see how your changes might impact your running resources, especially for critical resources, before implementing them.

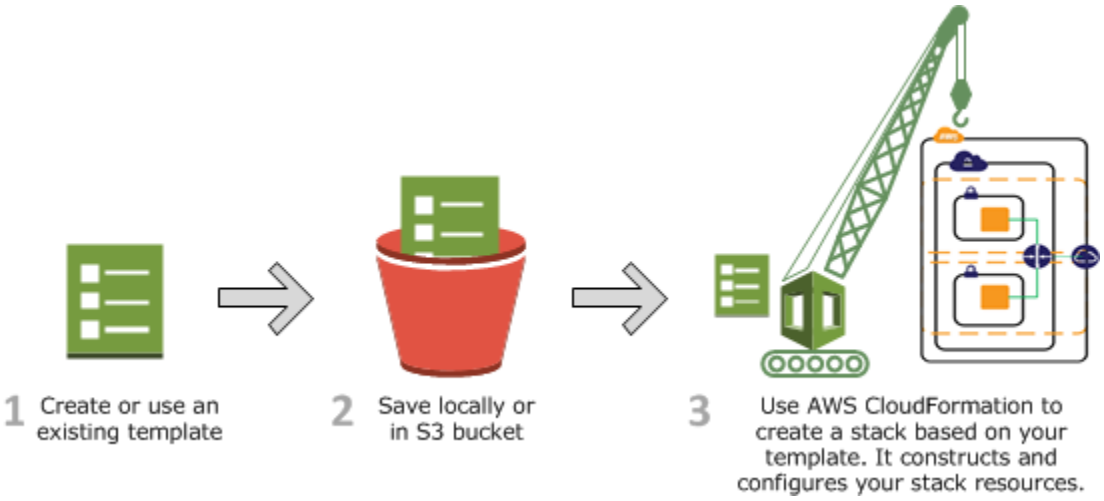
For example, if you change the name of an Amazon RDS database instance, CloudFormation will create a new database and delete the old one. You will lose the data in the old database unless you've already backed it up. If you generate a change set, you will see that your change will cause your database to be replaced, and you will be able to plan accordingly before you update your stack.

How CloudFormation works

When you use CloudFormation to create your stack, CloudFormation makes underlying service calls to AWS to provision and configure the resources described in your template. You need permission to create these resources. For example, to create EC2 instances by using CloudFormation, you need permissions to create instances. You manage these permissions with AWS Identity and Access Management (IAM).

The calls that CloudFormation makes are all declared by your template. For example, suppose you have a template that describes an EC2 instance with a `t2.micro` instance type. When you use that

template to create a stack, CloudFormation calls the Amazon EC2 create instance API and specifies the instance type as `t2.micro`. The following diagram summarizes the CloudFormation workflow for creating stacks.



To create a stack

1. Use a text editor to create a CloudFormation template in YAML or JSON format. The CloudFormation template describes the resources you want and their settings. Use Infrastructure Composer to visualize and validate your template. This helps you make sure that your template is properly structured and free of syntax errors. For more information, see Working with CloudFormation templates.
2. Save the template locally or in an Amazon S3 bucket.
3. Create a CloudFormation stack by specifying the location of your template file, such as a path on your local computer or an Amazon S3 URL. If the template contains parameters, you can specify input values when you create the stack. Parameters allow you to pass in values to your template so that you can customize your resources each time you create a stack.

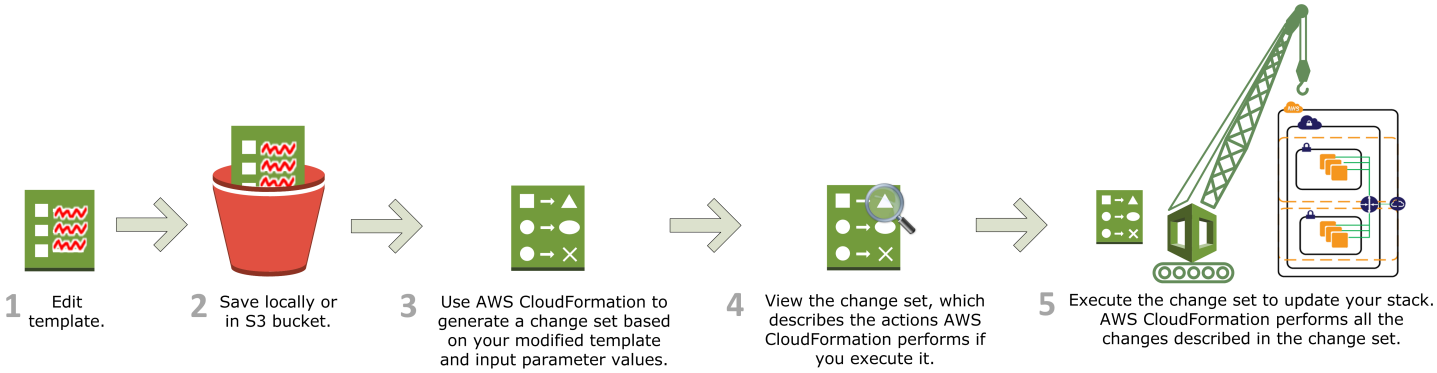
> **Note**
>
> If you specify a template file stored locally, CloudFormation uploads it to an S3 bucket in your AWS account. CloudFormation creates a bucket for each region in which you upload a template file. The buckets are accessible to anyone with Amazon Simple Storage Service (Amazon S3) permissions in your AWS account. If a bucket created by CloudFormation is already present, the template is added to that bucket.

> You can use your own bucket and manage its permissions by manually uploading templates to Amazon S3. Then whenever you create or update a stack, specify the Amazon S3 URL of a template file.

After all the resources have been created, CloudFormation reports that your stack has been created. You can then start using the resources in your stack. If stack creation fails, CloudFormation rolls back your changes by deleting the resources that it created.

Updating a stack with a change set

When you need to update your stack's resources, you can modify the stack's template. You don't need to create a new stack and delete the old one. To update a stack, create a change set by submitting a modified version of the original stack template, different input parameter values, or both. CloudFormation compares the modified template with the original template and generates a change set. The change set lists the proposed changes. After reviewing the changes, you can start the change set to update your stack or you can create a new change set. The following diagram summarizes the workflow for updating a stack.



To update a stack with a change set

1. You can modify a CloudFormation stack template by using Infrastructure Composer or a text editor. For more information, see Update your stack template.

   As you update your template, keep in mind that updates can cause interruptions. Depending on the resource and properties that you are updating, an update might interrupt or even replace an existing resource. For more information, see Understand update behaviors of stack resources.

2. Save the CloudFormation template locally or in an S3 bucket.

3. Create a change set by specifying the stack that you want to update and the location of the modified template, such as a path on your local computer or an Amazon S3 URL. For more information about creating change sets, see Update CloudFormation stacks using change sets.

> **Note**
>
> If you specify a template that's stored on your local computer, CloudFormation automatically uploads your template to an S3 bucket in your AWS account.

4. View the change set to check that CloudFormation will perform the changes that you expect. For example, check whether CloudFormation will replace any critical stack resources. You can create as many change sets as you need until you have included the changes that you want.

> **Important**
>
> Change sets don't indicate whether your stack update will be successful. For example, a change set doesn't check if you will surpass an account quota, if you're updating a resource that doesn't support updates, or if you have insufficient permissions to modify a resource, which can cause a stack update to fail.

5. Initiate the change set that you want to apply to your stack. CloudFormation updates your stack by updating only the resources that you modified and signals that your stack has been successfully updated. If the stack updates fails, CloudFormation rolls back changes to restore the stack to the last known working state.

Ways to get started with CloudFormation

To create a hello world CloudFormation stack with the console, see Creating your first stack.

For guided learning, try the Getting Started with CloudFormation workshop, which offers hands-on experience with template development.

Signing up for an AWS account

When you sign up for AWS, your AWS account is automatically signed up for all services in AWS, including CloudFormation. If you have an AWS account already, skip to the next topic. If you don't have an AWS account, use the following procedure to create one.

Sign up for an AWS account

If you do not have an AWS account, complete the following steps to create one.

To sign up for an AWS account

1. Open https://portal.aws.amazon.com/billing/signup.
2. Follow the online instructions.

   Part of the sign-up procedure involves receiving a phone call or text message and entering a verification code on the phone keypad.

   When you sign up for an AWS account, an *AWS account root user* is created. The root user has access to all AWS services and resources in the account. As a security best practice, assign administrative access to a user, and use only the root user to perform tasks that require root user access.

AWS sends you a confirmation email after the sign-up process is complete. At any time, you can view your current account activity and manage your account by going to https://aws.amazon.com/ and choosing **My Account**.

Create a user with administrative access

After you sign up for an AWS account, secure your AWS account root user, enable AWS IAM Identity Center, and create an administrative user so that you don't use the root user for everyday tasks.

Secure your AWS account root user

1. Sign in to the AWS Management Console as the account owner by choosing **Root user** and entering your AWS account email address. On the next page, enter your password.

   For help signing in by using root user, see Signing in as the root user in the *AWS Sign-In User Guide*.
2. Turn on multi-factor authentication (MFA) for your root user.

   For instructions, see Enable a virtual MFA device for your AWS account root user (console) in the *IAM User Guide*.

Create a user with administrative access

1. Enable IAM Identity Center.

   For instructions, see Enabling AWS IAM Identity Center in the *AWS IAM Identity Center User Guide*.

2. In IAM Identity Center, grant administrative access to a user.

   For a tutorial about using the IAM Identity Center directory as your identity source, see Configure user access with the default IAM Identity Center directory in the *AWS IAM Identity Center User Guide*.

Sign in as the user with administrative access

- To sign in with your IAM Identity Center user, use the sign-in URL that was sent to your email address when you created the IAM Identity Center user.

  For help signing in using an IAM Identity Center user, see Signing in to the AWS access portal in the *AWS Sign-In User Guide*.

Assign access to additional users

1. In IAM Identity Center, create a permission set that follows the best practice of applying least-privilege permissions.

   For instructions, see Create a permission set in the *AWS IAM Identity Center User Guide*.

2. Assign users to a group, and then assign single sign-on access to the group.

   For instructions, see Add groups in the *AWS IAM Identity Center User Guide*.

> **Note**
>
> For more information on how to manage who has access to what, see Control CloudFormation access with AWS Identity and Access Management.

Creating your first stack

This topic walks you through creating your first CloudFormation stack using the AWS Management Console. By following this tutorial, you'll learn how to provision basic AWS resources, monitor stack events, and generate outputs.

For this example, the CloudFormation template is written in YAML. YAML is a human-readable format that's widely used for defining infrastructure as code. As you learn more about CloudFormation, you might also encounter other templates in JSON format, but for this tutorial, YAML is chosen for its readability.

> **Note**
>
> CloudFormation is free, but you'll be charged for the Amazon EC2 and Amazon S3 resources you create. However, if you're new to AWS, you can take advantage of the Free Tier to minimize or eliminate costs during this learning process.

Topics

- Prerequisites
- Create a CloudFormation stack with the console
- Monitor stack creation
- Test the web server
- Troubleshooting
- Clean up
- Next steps

Prerequisites

- You must have access to an AWS account with an IAM user or role that has permissions to use Amazon EC2, Amazon S3, and CloudFormation, or administrative user access.
- You must have a Virtual Private Cloud (VPC) that has access to the internet. This walkthrough template requires a default VPC, which comes automatically with newer AWS accounts. If you don't have a default VPC, or if it was deleted, see the troubleshooting section in this topic for alternative solutions.

Create a CloudFormation stack with the console

To create a Hello world CloudFormation stack with the console

1. Open the CloudFormation console.
2. Choose **Create Stack**.
3. On the **Create stack** page, choose **Build from Infrastructure Composer**, and then **Create in Infrastructure Composer**. This takes you to Infrastructure Composer in CloudFormation console mode where you can upload and validate the example template.
4. To upload and validate the example template, do the following:

   a. Choose **Template**. Then, copy and paste the following CloudFormation template into the template editor:

```
AWSTemplateFormatVersion: 2010-09-09
Description: CloudFormation Template for WebServer with Security Group and EC2
 Instance

Parameters:
  LatestAmiId:
    Description: The latest Amazon Linux 2 AMI from the Parameter Store
    Type: AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>
    Default: '/aws/service/ami-amazon-linux-latest/amzn2-ami-hvm-x86_64-gp2'

  InstanceType:
    Description: WebServer EC2 instance type
    Type: String
    Default: t2.micro
    AllowedValues:
      - t3.micro
      - t2.micro
    ConstraintDescription: must be a valid EC2 instance type.

  MyIP:
    Description: Your IP address in CIDR format (e.g. 203.0.113.1/32).
    Type: String
    MinLength: '9'
    MaxLength: '18'
    Default: 0.0.0.0/0
    AllowedPattern: '^(\d{1,3}\.){3}\d{1,3}\/\d{1,2}$'
    ConstraintDescription: must be a valid IP CIDR range of the form x.x.x.x/x.
```

```
Resources:
  WebServerSecurityGroup:
    Type: AWS::EC2::SecurityGroup
    Properties:
      GroupDescription: Allow HTTP access via my IP address
      SecurityGroupIngress:
        - IpProtocol: tcp
          FromPort: 80
          ToPort: 80
          CidrIp: !Ref MyIP

  WebServer:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: !Ref LatestAmiId
      InstanceType: !Ref InstanceType
      SecurityGroupIds:
        - !Ref WebServerSecurityGroup
      UserData: !Base64 |
        #!/bin/bash
        yum update -y
        yum install -y httpd
        systemctl start httpd
        systemctl enable httpd
        echo "<html><body><h1>Hello World!</h1></body></html>" > /var/www/html/
index.html

Outputs:
  WebsiteURL:
    Value: !Join
      - ''
      - - http://
        - !GetAtt WebServer.PublicDnsName
    Description: Website URL
```

Before you move to the next step, let's take a moment to take a look at the template and understand some key CloudFormation concepts.

- The **`Parameters`** section declares values that can be passed to the template when you create the stack. Resources specified later in the template reference these values and use the data. Parameters are an effective way to specify information that you don't

want to store in the template itself. They're also a way to specify information that might be unique to the specific application or configuration you are deploying.

- The template defines the following parameters:
  - **`LatestAmiId`** – Retrieves the latest Amazon Linux 2 AMI ID from the AWS Systems Manager Parameter Store.
  - **`InstanceType`** – Allows selection of the EC2 instance type (default: `t2.micro`, allowed: `t3.micro`, `t2.micro`).
  - **`MyIP`** – Specifies the IP address range for HTTP access (default: 0.0.0.0/0, allowing access from any IP).
- The **`Resources`** section contains the definitions of the AWS resources you want to create with the template. Resource declarations are an efficient way to specify all these configuration settings at once. When you include resource declarations in a template, you can create and configure all the declared resources by using that template to create a stack. You can also create new stacks from the same template to launch identical configurations of resources.
- This template creates the following resources:
  - **`WebServerSecurityGroup`** – An EC2 security group that allows inbound HTTP traffic on port 80 from the specified IP range.
  - **`WebServer`** – An EC2 instance with the following configuration:
    - Uses the latest Amazon Linux 2 AMI
    - Applies the selected instance type
    - Adds the `WebServerSecurityGroup` to the `SecurityGroupIds` property
    - Includes a user data script to install Apache HTTP Server
- A logical name is specified at the beginning of each resource and parameter declaration. For example, `WebServerSecurityGroup` is the logical name assigned to the EC2 security group resource. The `Ref` function is then used to reference resources and parameters by their logical names in other parts of the template. When one resource references another resource, this creates a dependency between them.
- The **`Outputs`** section defines custom values that are returned after the stack creation. You can use the output values to return information from the resources in the stack, such as resource identifiers or URLs.
- The template defines one output:

- **`WebsiteURL`** – The URL of the deployed web server, constructed using the EC2 instance's public DNS name. The `Join` function helps combine the fixed `http://` with the variable `PublicDnsName` into a single string, making it easy to output the full URL of the web server.

b. Choose **Validate** to make sure the YAML code is valid before uploading the template.

c. Next, choose **Create template** to create the template and add it to an S3 bucket.

d. From the dialog box that opens, make a note of the name of the S3 bucket so you can delete it later. Then, choose **Confirm and continue to CloudFormation**. This takes you to the CloudFormation console where the S3 path to your template is now specified.

5. On the **Create stack** page, choose **Next**.
6. On the **Specify stack details** page, type a name in the **Stack name** field. The stack name can't contain spaces. For this example, use **`MyTestStack`**.
7. Under **Parameters**, specify parameter values as follows:

   - **LatestAmiId**: This is set by default to the latest Amazon Linux 2 AMI.
   - **InstanceType**: Choose either **t2.micro** or **t3.micro** for the EC2 instance type.

     > **Note**
     >
     > If you're new to AWS, you can use the free tier to launch and use a `t2.micro` instance for free for 12 months (in Regions where `t2.micro` is unavailable, you can use a `t3.micro` instance under the free tier).

   - **MyIP**: Specify your actual public IP address with a `/32` suffix. The `/32` suffix is used in CIDR notation to specify that a single IP address is allowed. It essentially means allow traffic to and from this specific IP address, and no others.
8. Choose **Next** twice to go to the **Review and create** page. For this tutorial, you can leave the defaults on the **Configure stack options** page as they are.
9. Review the information for the stack. When you're satisfied with the settings, choose **Submit**.

Monitor stack creation

After you choose **Submit**, CloudFormation begins creating the resources that are specified in the template. Your new stack, **`MyTestStack`**, appears in the list at the top portion of the

CloudFormation console. Its status should be `CREATE_IN_PROGRESS`. You can see detailed status for a stack by viewing its events.

To view the events for the stack

1. On the CloudFormation console, choose the stack **`MyTestStack`** in the list.
2. In the stack details pane, choose the **Events** tab.

   The console automatically refreshes the event list with the most recent events every 60 seconds.

The **Events** tab displays each major step in the creation of the stack sorted by the time of each event, with latest events on top.

The first event (at the bottom of the event list) is the start of the stack creation process:

```
2024-12-23 18:54 UTC-7 MyTestStack CREATE_IN_PROGRESS User initiated
```

Next are events that mark the beginning and completion of the creation of each resource. For example, creation of the EC2 instance results in the following entries:

```
2024-12-23 18:59 UTC-7 WebServer CREATE_COMPLETE
```

```
2024-12-23 18:54 UTC-7 WebServer CREATE_IN_PROGRESS Resource creation
initiated
```

The `CREATE_IN_PROGRESS` event is logged when CloudFormation reports that it has begun to create the resource. The `CREATE_COMPLETE` event is logged when the resource is successfully created.

When CloudFormation has successfully created the stack, you will see the following event at the top of the **Events** tab:

```
2024-12-23 19:17 UTC-7 MyTestStack CREATE_COMPLETE
```

If CloudFormation can't create a resource, it reports a `CREATE_FAILED` event and, by default, rolls back the stack and deletes any resources that have been created. The **Status Reason** column displays the issue that caused the failure.

After the stack is created, you can go to the **Resources** tab to view the EC2 instance and security group you created.

Test the web server

After the stack is successfully created, navigate to the **Outputs** tab in the CloudFormation console. Look for the **WebsiteURL** field. This will contain the public URL of your EC2 instance.

Open a browser and go to the URL listed under **WebsiteURL**. You should see a simple "Hello World!" message displayed in the browser.

This confirms that your EC2 instance is running Apache HTTP Server and serving a basic web page.

Troubleshooting

If you experience a rollback during stack creation, it might be due to a missing VPC. Here's how to resolve this issue.

No default VPC available

The template in this walkthrough requires a default VPC. If your stack creation fails because of VPC or subnet availability errors, you might not have a default VPC in your account. You have the following options:

- **Create a new default VPC** – You can create a new default VPC through the Amazon VPC console. For instructions, see Create a default VPC in the *Amazon VPC User Guide*.
- **Modify the template to specify a subnet** – If you have a non-default VPC, you can modify the template to explicitly specify the VPC and subnet IDs. Add the following parameter to the template:

  ```
  SubnetId:
    Description: The subnet ID to launch the instance into
    Type: AWS::EC2::Subnet::Id
  ```

  Then, update the `WebServer` resource to include the subnet ID:

  ```
  WebServer:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: !Ref LatestAmiId
      InstanceType: !Ref InstanceType
      SecurityGroupIds:
        - !Ref WebServerSecurityGroup
  ```

```
      SubnetId: !Ref SubnetId
      UserData: !Base64 |
        #!/bin/bash
        yum update -y
        yum install -y httpd
        systemctl start httpd
        systemctl enable httpd
        echo "<html><body><h1>Hello World!</h1></body></html>" > /var/www/html/
index.html
```

When creating the stack, you'll need to specify a subnet that has internet access for the web server to be reachable.

Clean up

To make sure you aren't charged for any unwanted services, you can clean up by deleting the stack and its resources. You can also delete the S3 bucket that stores the stack's template.

To delete the stack and its resources

1. Open the CloudFormation console.
2. On the **Stacks** page, select the option next to the name of the stack you created (**`MyTestStack`**) and then choose **Delete**.
3. When prompted for confirmation, choose **Delete**.
4. Monitor the progress of the stack deletion process on the **Event** tab. The status for **`MyTestStack`** changes to `DELETE_IN_PROGRESS`. When CloudFormation completes the deletion of the stack, it removes the stack from the list.

If you are done working with the example template and no longer need your Amazon S3 bucket, delete it. Before you can delete a bucket, you must first empty it. Emptying a bucket deletes all objects in it.

To empty and delete the Amazon S3 bucket

1. Open the Amazon S3 console.
2. In the navigation pane on the left side of the console, choose **Buckets**.
3. In the **Buckets** list, select the option next to the name of the bucket that you created for this tutorial, and then choose **Empty**.

4. On the **Empty bucket** page, confirm that you want to empty the bucket by typing **`permanently delete`** into the text field, and then choose **Empty**.
5. Monitor the progress of the bucket emptying process on the **Empty bucket: Status** page.
6. To return to your bucket list, choose **Exit**.
7. Select the option next to the name of the bucket, and then choose **Delete**.
8. When prompted for confirmation, type the name of the bucket and then choose **Delete bucket**.
9. Monitor the progress of the bucket deletion process from the **Buckets** list. When Amazon S3 completes the deletion of the bucket, it removes the bucket from the list.

Next steps

Congratulations! You successfully created a stack, monitored its creations, and used its output.

To continue learning:

- Learn more about templates so that you can create your own. For more information, see Working with CloudFormation templates.
- Try the Getting Started with CloudFormation workshop for more hands-on practice with template creation.
- For a shortened version of Getting Started with CloudFormation, see Deploy applications on Amazon EC2. This topic describes the same scenario of using a CloudFormation helper script, `cfn-init`, to bootstrap an Amazon EC2 instance.